

WEEK3- MULTITHREADED PROGRAMMING



Multithreaded Programming

- A multithreaded program contains two or more parts that can run concurrently.
- Each part of a multithreaded program is called a thread.
- Each thread defines a separate path of execution.
- Java provides built in support for multithreaded programming.
- Multithreading is a specialized form of multitasking.

- The two types of multitasking are

1. Process-based
2. Thread based

Process-based multitasking :

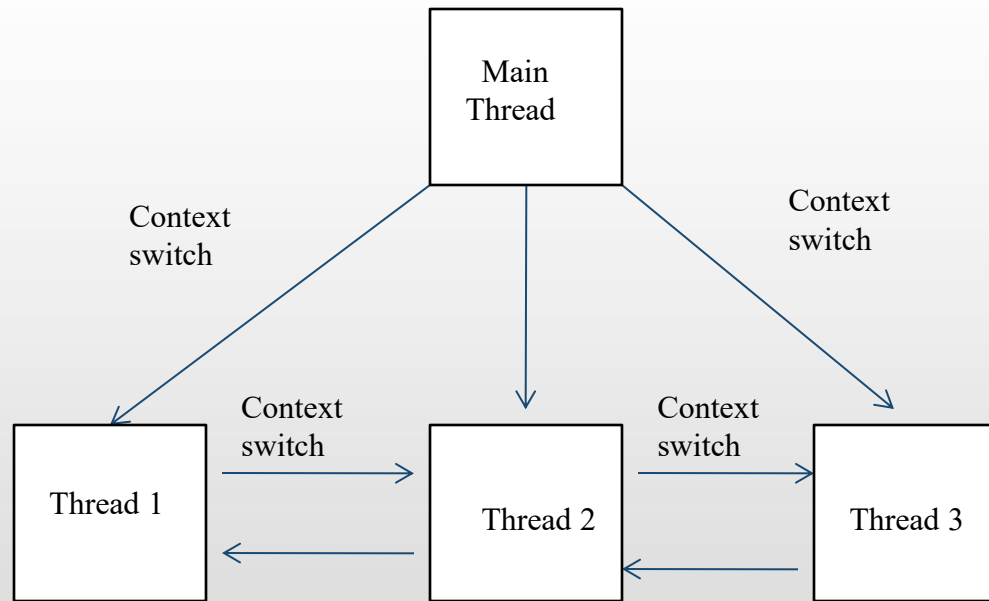
- allows computer to run two or more programs concurrently.
- **For ex:** run the Java compiler at the same time using a text editor.

Thread based multitasking:

- Thread is the smallest unit of dispatchable code.
- **A single program can perform two or more tasks simultaneously.**
- **Ex:** a text editor can format text at the same time that is printing.

Process based	Thread based
More overhead	Less overhead
Processes are heavyweight tasks that require their own separate address spaces	Threads are lightweight processes and they share the same address space.
Context switching from one process to other process is costly.	context switching from one thread to the next is low cost.
Not under the control of java	Under the control of java

Multithreaded program



Multithreading enables us to write very efficient programs that make the maximum use of the CPU, because the idle time can be kept to a minimum.

The Thread class and the Runnable Interface

Multithreading in Java is facilitated using,

1. **Thread class** and its methods
 2. The interface **Runnable**
- To create a new thread our program will have to extend either the **Thread class** or implement the **Runnable interface**.

Methods defined by Thread class

Method	Meaning
getName	Obtain a thread's name.
getPriority	Obtain a thread's priority.
isAlive	Determine if a thread is still running.
join	Wait for a thread to terminate.
run	Entry point for the thread.
sleep	Suspend a thread for a period of time.
start	Start a thread by calling its run method.

The Main Thread

When a Java program is started the **Main thread runs immediately**. ie, it starts execution.

Importance of Main Thread:

1. It is the thread from which other “**child**” threads will be spawned.
2. Often, it must be the **last thread** to finish execution because it performs various shutdown actions.

The main thread can be controlled through a **Thread** object.

It is done by obtaining a reference to it by calling the method **currentThread()**

The `currentThread()` is a public static member of `Thread` .

General form :

```
static Thread currentThread()
```

It returns a reference to the thread in which it is called.

By using a reference to the main thread, we can control it like any other thread.

Example

// Controlling the main Thread.

```
class CurrentThreadDemo
```

```
{
    public static void main(String args[])
    {
        Thread t = Thread.currentThread();
        System.out.println("Current thread: " + t);
        t.setName("My Thread");
        System.out.println("After name change: " + t);
        try
        {
            for(int n = 5;n>0;n--)
            {
                System.out.println(n);
                Thread.sleep(1000);
            }
        } catch(InterruptedException e)
        {
            System.out.println("Main thread interrupted");
        }
    }
}
```

Current thread: Thread [**main**, 5, main]

After name change: Thread [**My Thread**, 5, main]

5

4

3

2

1

- The first line in the output displays the name of the thread, its priority and the name of its group.
- The second line displays the output after changing the thread name.
- Thread group is a data structure that controls the state of a collection of threads as a whole.
- The sleep() method causes the thread from which it is called to suspend execution for the specified period of milliseconds.

General form:

1. `static void sleep(long milliseconds) throws InterruptedException`
2. `static void sleep(long milliseconds, int nanoseconds) throws InterruptedException`

The second form allows us to specify the period in terms of milliseconds and nanoseconds.

We can set the name of a thread by using setName().

General form:

```
final void setName(String threadName)
```

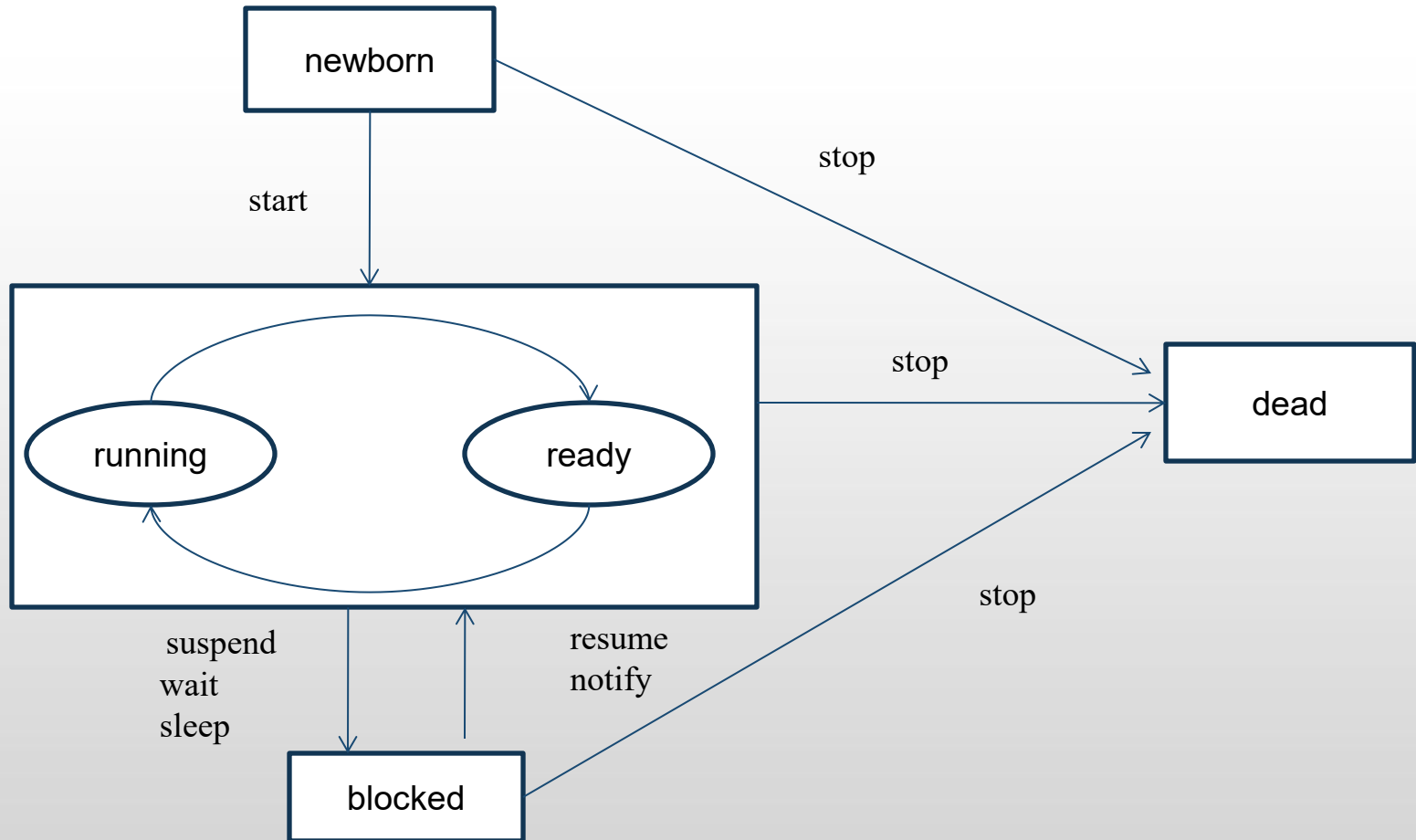
Here threadName specifies the name of the thread.

We can obtain the name of a thread by calling getName().

General form:

```
final String getName()
```

Life cycle of a thread



Creating a Thread

A thread can be created by **instantiating an object** of type Thread.

The 2 ways defined by Java for the creation of thread are:

1. By **Implementing the Runnable interface.**
2. By **Extending the Thread class.**

Implementing Runnable:

We can construct a thread on any object that implements **Runnable**.

To implement Runnable, a class need to implement a single method called **run()**.

General form:

```
public void run()
```

- Inside run(), we will define the code that constitutes the new thread.
- The run() can call other methods, use other classes and declare variables like the main thread.
- run() establishes an entry point for another concurrent thread of execution within our program.

After creating a class that implements Runnable ,we can instantiate an object of type Thread from within that class.

Constructors defined by Thread:

Thread()

Thread(Runnable threadOb)

Thread(String threadName)

Thread(Runnable threadOb, String threadName)

Here,

- threadOb is an instance of a class that implements Runnable interface. It defines where the execution of the thread will begin.
- threadName is the name of the new thread.

After the creation of a new thread it will not start running until the start() method declared inside the Thread is called.

start() executes a call to run().

General form:

```
void start()
```

```
class NewThread implements Runnable
```

```
{
    Thread t;
    NewThread()
    {
        t = new Thread(this);
        t.start(); // Start the thread
    }
    public void run()
    {
    }
}
```

```
class ThreadDemo
```

```
{
    public static void main(String args[])
    {
        NewThread t = new NewThread(); // create a new thread
    }
}
```

```
class NewThread implements Runnable
```

```
{
```

```
    Thread t;
```

```
    NewThread()
```

```
{        t = new Thread(this, "Demo Thread");
```

```
    System.out.println("Child thread: " + t);
```

```
    t.start(); // Start the thread
```

```
}
```

```
// This is the entry point for the second thread
```

```
public void run()
```

```
{    try {
```

```
        for(int i = 5; i > 0; i--)
```

```
        {            System.out.println("Child Thread: " + i);
```

```
            Thread.sleep(500);
```

```
        }
```

```
    }
```

```
    catch (InterruptedException e)
```

```
    {        System.out.println("Child interrupted.");
```

```
    }
```

```
    System.out.println("Exiting child thread.");
```

```
}
```

```
}
```

```
class ThreadDemo
{
    public static void main(String args[])
    {
        new NewThread(); // create a new thread
        try {
            for(int i = 5; i > 0; i--)
            {
                System.out.println("Main Thread: " + i);
                Thread.sleep(1000);
            }
        }
        catch (InterruptedException e)
        {
            System.out.println("Main thread interrupted.");
        }
        System.out.println("Main thread exiting.");
    }
}
```

A new thread is created by the following statement:

```
t = new Thread(this, "Demo Thread");
```

Extending Thread

- A class that **extends Thread** is another way of **creating a thread** and **creating an instance** of that class.
- The **extending class** must **override the run()** method, which is the entry point for the new thread.
- It must also call the **start()** method to **begin the execution** of the **new thread**.

// Create a second thread by extending Thread

```
class NewThread extends Thread
```

```
{    NewThread()
```

```
    {        super("Demo Thread");
```

```
        System.out.println("Child thread: " + this);
```

```
        start(); // Start the thread
```

```
    }
```

```
    public void run()
```

```
    {        try {            for(int i = 5; i > 0; i--)
```

```
                {        System.out.println("Child Thread: " + i);
```

```
                    Thread.sleep(500);
```

```
                }
```

```
            }
```

```
            catch (InterruptedException e)
```

```
            {        System.out.println("Child interrupted.");
```

```
            }
```

```
            System.out.println("Exiting child thread.");
```

```
        }
```

```
    }
```

```

class ExtendThread
{
    public static void main(String args[])
    {
        new NewThread(); // create a new thread
        try
        {
            for(int i = 5; i > 0; i--)
            {
                System.out.println("Main Thread: " + i);
                Thread.sleep(1000);
            }
        }
        catch (InterruptedException e)
        {
            System.out.println("Main thread interrupted.");
        }
        System.out.println("Main thread exiting.");
    }
}

```

Here the call to **super()** inside **NewThread** invokes the following form of the **Thread constructor**

```
public Thread(String threadName)
```

Creating multiple threads

```
class NewThread implements Runnable
{
    String name; // name of thread
    Thread t;
    NewThread(String threadname)
    {
        name = threadname;
        t = new Thread(this, name);
        System.out.println("New thread: " + t);
        t.start(); // Start the thread
    }
    public void run()
    {
        try
        {
            for(int i = 5; i > 0; i--)
            {
                System.out.println(name + ": " + i);
                Thread.sleep(1000);
            }
        }
        catch (InterruptedException e)
        {
            System.out.println(name + "Interrupted");
        }
        System.out.println(name + " exiting.");
    }
}
```

```
class MultiThreadDemo
{
    public static void main(String args[])
    {
        new Thread("One"); // start threads
        new Thread("Two");
        new Thread("Three");
        try
        {
            // wait for other threads to end
            Thread.sleep(10000);
        }
        catch (InterruptedException e)
        {
            System.out.println("Main thread Interrupted");
        }
        System.out.println("Main thread exiting.");
    }
}
```

```
class MyThread extends Thread
```

```
{  
    public void run()  
    {  
  
    }  
}
```

```
class prg
```

```
{  
    public static void main(String args[])  
    {  
        MyThread t = new MyThread();  
        t.start();  
    }  
}
```

```
class MyThread extends Thread
```

```
{  
    MyThread()  
    {  
        start();  
    }  
    public void run()  
    {  
  
    }  
}
```

```
class prg
```

```
{  
    public static void main(String args[])  
    {  
        MyThread t = new MyThread();  
    }  
}
```

```

class MyThread implements Runnable
{
    public void run()
    {

    }
}
class prg
{
    public static void main(String args[])
    {
        MyThread t = new MyThread();
        Thread t1 = new Thread(t);
        t1.start();
    }
}

```

```

class MyThread implements Runnable
{
    Thread t
    MyThread()
    {
        t = new Thread(this);
        t.start();
    }
    public void run()
    {

    }
}
class prg
{
    public static void main(String args[])
    {
        MyThread t = new MyThread();
    }
}

```

Using `isAlive()` and `join()`

Two ways to determine whether a thread has finished or not are:

1. By calling `isAlive()` method defined by **Thread** on the thread.

General form:

```
final boolean isAlive()
```

It **returns true** if the thread upon which it is called is **running**, else **returns false**.

2. By calling the method `join()`

General form:

```
final void join() throws InterruptedException
```

This method waits until the thread on which it is called terminates.

// Using join() to wait for threads to finish.

```
class NewThread extends Thread
{
    String name; // name of thread

    NewThread(String threadname)
    {
        name = threadname;
        System.out.println("New thread: " + t);
        start();
    }

    public void run()
    {
        try
        {
            for(int i = 5; i > 0; i--)
            {
                System.out.println(name + ": " + i);
                Thread.sleep(1000);
            }
        }
        catch (InterruptedException e)
        {
            System.out.println(name + " interrupted.");
        }
        System.out.println(name + " exiting.");
    }
}
```

```
class DemoJoin
```

```
{    public static void main(String args[])
```

```
{        NewThread ob1 = new NewThread("One");
        NewThread ob2 = new NewThread("Two");
        NewThread ob3 = new NewThread("Three");
```

```
        System.out.println("Thread One is alive: " + ob1.isAlive());
        System.out.println("Thread Two is alive: " + ob2.isAlive());
        System.out.println("Thread Three is alive: " + ob3.isAlive());
```

```
        try
```

```
{            System.out.println("Waiting for threads to finish.");
            ob1.join();
            ob2.join();
            ob3.join();
```

```
        }
```

```
        catch (InterruptedException e) {            System.out.println("Main thread Interrupted");        }
```

```
        System.out.println("Thread One is alive: " + ob1.isAlive() );
        System.out.println("Thread Two is alive: " + ob2.isAlive() );
        System.out.println("Thread Three is alive: " + ob3.isAlive() );
        System.out.println("Main thread exiting.");    } }
```

Thread priorities

- Used by the thread scheduler to decide when each thread should be allowed to run.
- Higher priority threads get more CPU time than lower priority threads(Theoritically).
- To set a thread's priority the method **setPriority()** which is a member of **Thread** is used.

General form:

```
final void setPriority(int level)
```

value of level should be within the **MIN_PRIORITY** and **MAX_PRIORITY**. ie, between **1 to 10**.

To return a thread to **default priority**, specify **NORM_PRIORITY**, which is currently **5**.

- To get the value of current priority setting we can call the **getPriority()** method defined by the **Thread**.

General form:

```
final int getPriority()
```

```
class MyThread extends Thread
{
    String name;
    MyThread(String s)
    {    this.name = s;    }
    public void run()
    {
        for(int i=1;i<=500;i++)
            System.out.println("Thread "+name);
        System.out.println("End of Thread "+name);
    }
}

class ThreadPrg
{    public static void main(String args[]) {
        MyThread t1 = new MyThread("First Thread");
        MyThread t2 = new MyThread("Second Thread");
        MyThread t3 = new MyThread("Third Thread");

        t1.setPriority(Thread.MAX_PRIORITY);
        t2.setPriority(t2.getPriority()+1);
        t3.setPriority(Thread.MIN_PRIORITY);
        t1.start();
        t2.start();
        t3.start();    }}
}
```